



Versionierung mit Git: die Basics

Lokale Versionskontrolle für JavaScript-Widgets

14.–16. September 2026 · Universität Mozarteum Salzburg

Dr. Adam G. Augustin · FM meets Mozart

Folien wurden unter Zuhilfenahme von KI erstellt.

Über mich

Selbständiger FileMaker Entwickler im Raum München

Beratung und Entwicklung seit über 15 Jahren

Entwicklung von kundenspezifischen Datenbanken sowie
Betreuung und Weiterentwicklung bestehender Lösungen

Web- und App-Entwicklung

FileMaker zertifizierter Partner

Zahlreiche Vorträge auf der FMK, dotfmp und vienna calling

Mehr zu meinen Projekten mit Arbeitsbeispielen auf
www.agametis.de und <https://github.com/agametis>

FileMaker Magazin Award 2024 Gewinner





Was wir anschließend sicher können

- Änderungen in verständlichen Commits sichern
- Zwischen Branches wechseln und gezielt zusammenführen
- Den passenden Rückweg wählen — ohne unnötigen Datenverlust



Ein Widget verändert sich schneller als gedacht

- HTML, CSS und JavaScript entwickeln sich gemeinsam
- Ein Versuch soll den stabilen Stand nicht gefährden
- Git macht Zwischenstände benennbar, vergleichbar und wiederherstellbar

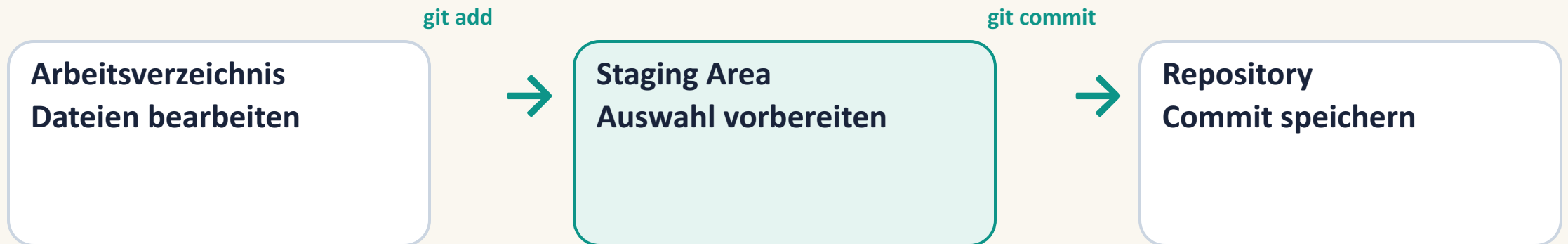


Repository initialisieren und Dateien ausschließen

- `git init` — Erstellt im aktuellen Ordner ein neues lokales Git-Repository und legt dafür den versteckten Ordner `.git` an.
- `.gitignore` — Legt fest, welche Dateien und Ordner Git nicht erfassen soll, beispielsweise `node_modules`, Build-Dateien oder lokale Konfigurationen.



Git kennt drei Arbeitszustände



git status zeigt jederzeit, wo eine Änderung steht.

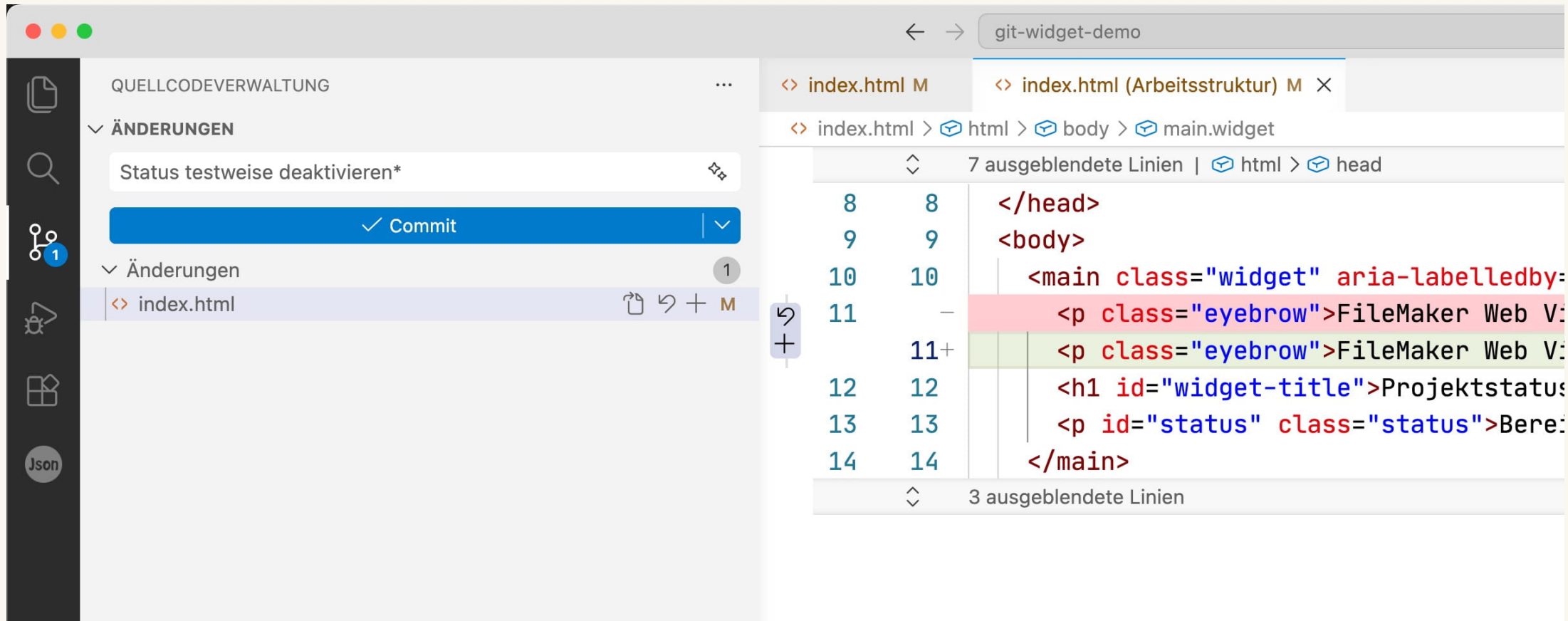


Ein Commit ist eine verständliche Zwischenstation

- Eine in sich geschlossene Änderung
- Eine kurze, konkrete Botschaft: Was wurde erreicht?
- Vorher Diff prüfen — erst dann committen



Alles beginnt in der Quellcodeverwaltung





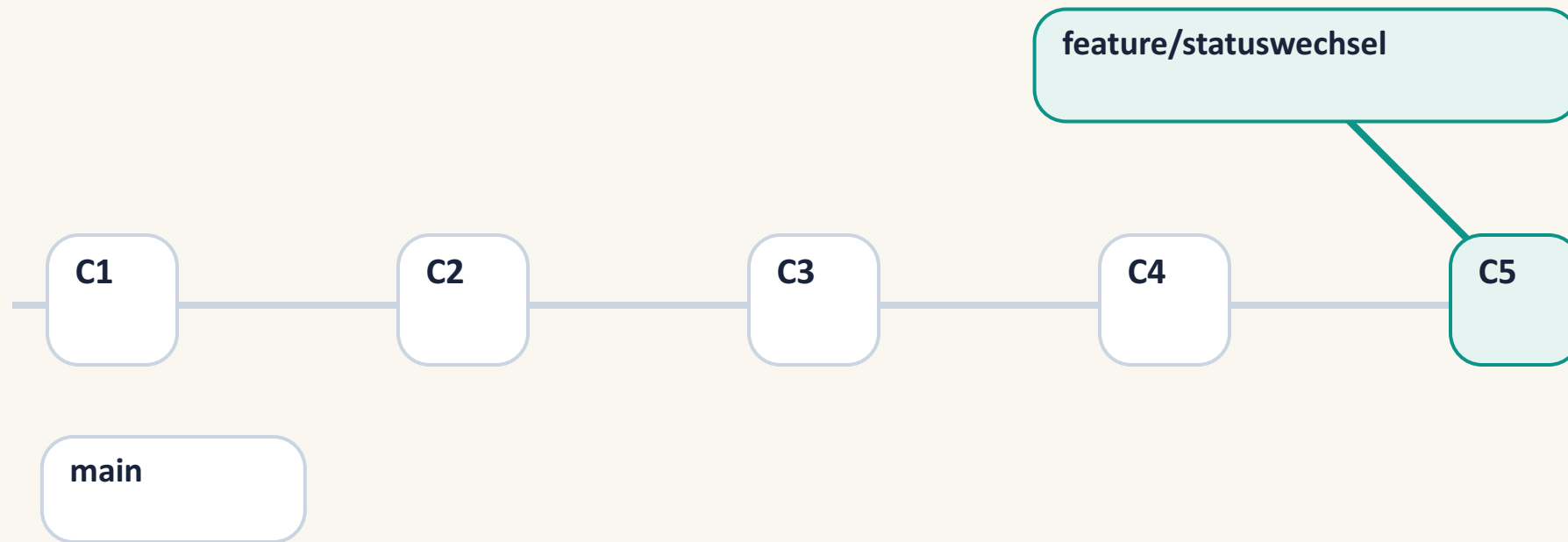
Dr. Adam G. Augustin

Branches

Neue Ideen isolieren, sicher wechseln und gezielt zusammenführen.



Branches sind bewegliche Zeiger



Commits bleiben; nur die Branch-Zeiger bewegen sich.



Branches sind bewegliche Zeiger

The screenshot shows the Git Graph view in VS Code. At the top, there are tabs for 'index.html M' and 'index.html (Arbeitsstruktur) M', followed by the 'Git Graph' tab. Below the tabs, there is a 'Branches:' dropdown menu set to 'Show All'. The main area displays a vertical timeline of commits and branches. The 'main' branch is highlighted in blue. Other branches shown include 'demo/revert', 'demo1-start', 'reserve/rueckgaengig', 'demo2-fehler', 'demo2-start', 'reserve/statuswechsel', 'demo1-fertig', and 'demo-basis'. Each branch is represented by a colored circle and a label, with a description of the commit it points to.

Gra...	Description	
Uncommitted Changes (1)		
demo/revert	Revert "Status vorübergehend deaktivieren"	
main	Anleitung für die Vortragsdemos ergänzen	
reserve/rueckgaengig	demo2-fehler	Status vorübergehend deaktivieren
demo/rueckgaengig	demo2-start	Hinweis zur lokalen Statusänderung ergänzen
reserve/statuswechsel	demo1-fertig	Statuswechsel im Widget ergänzen
demo-basis		Grundgerüst für Status-Widget anlegen

Graphische Darstellung des Git Graphen in VS Code

The screenshot shows the VS Code Extensions Marketplace. The search bar at the top contains the text 'Nach Erweiterungen in Marketplace suchen'. Below the search bar, there is a section for 'INSTALLIERT' (Installed) extensions. The 'Git Graph' extension by mhutchie is listed and marked as installed. Other extensions shown include 'German Language Pack for Visual Studio Code Deutsch' by Microsoft, 'json' by ZainChen, and 'Live Server'.

Extension Name	Author	Status
German Language Pack for Visual Studio Code Deutsch	Microsoft	Installed
Git Graph	mhutchie	Installed
json	ZainChen	Installed
Live Server		Installed

Mit installierter Erweiterung "Git Graph"



Branch erstellen oder wechseln

Terminal

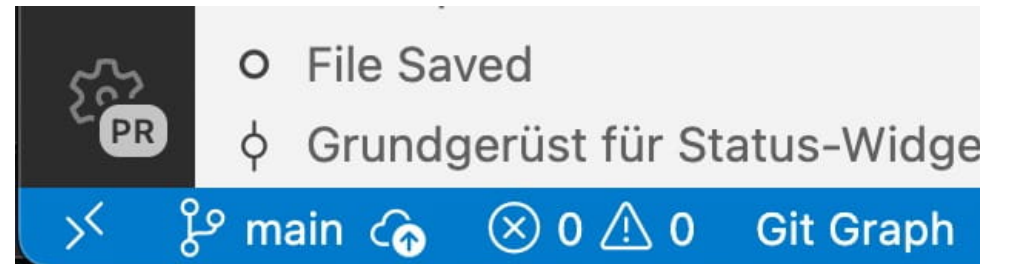
```
git switch -c <neuer-branch-name>
```

```
git switch main
```

VS Code

Branchname links unten anklicken

- Neuen Branch erstellen...
- Zielbranch auswählen





Der effizienteste Branch-Wechsel

Schnell zurück

`git switch -`

Wechselt zum zuletzt aktiven Branch.

Vor dem Wechsel

1. Änderungen prüfen
2. Committed oder bewusst stehen lassen
3. Branch links unten wählen (VS Code)



Der effizienteste Branch-Wechsel





Mergen: erst Ziel, dann Quelle

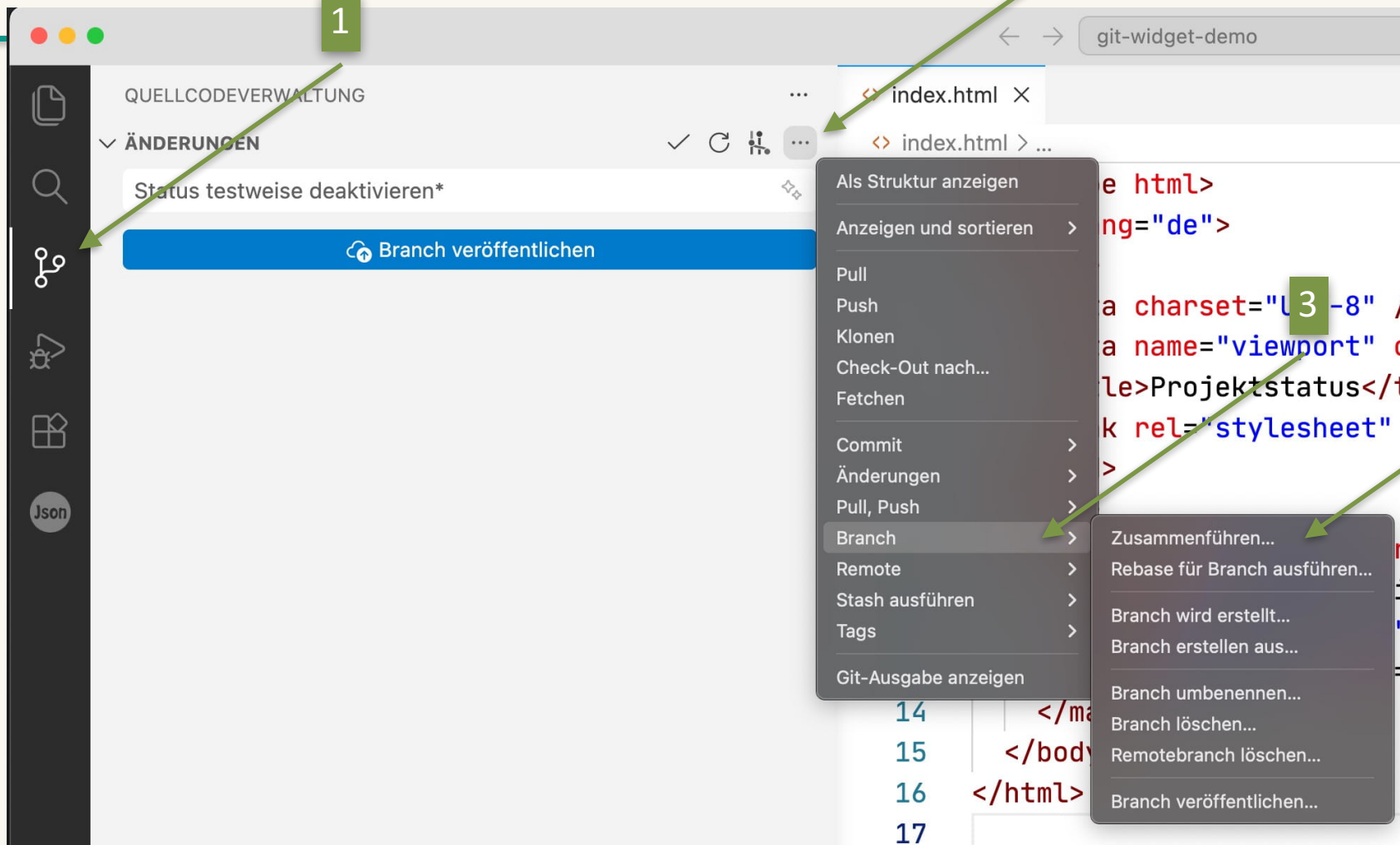
Terminal

```
git switch main  
git merge <branch-name>
```

VS Code

1. Zielbranch links unten wählen
2. Quellcodeverwaltung: ⋮
3. Branch → Zusammenführen...
4. Quellbranch auswählen

Mergen: erst Ziel, dann Quelle





Ein Konflikt ist eine Rückfrage, kein Defekt

- Beide Branches änderten dieselbe Stelle unterschiedlich
- VS Code zeigt aktuelle und eingehende Änderung im Editor
- Entscheidung treffen, Datei speichern, bereitstellen und committen



Dr. Adam G. Augustin

Änderungen zurücknehmen

Nicht jeder Rückweg meint dasselbe: verwerfen, entstagen, zurücksetzen oder rückgängig committen.



Die wichtigste Frage: Was soll erhalten bleiben?

Wo liegt die Änderung?

Noch nicht committed

restore / unstage

Letzter lokaler Commit

reset

Historie soll sichtbar bleiben

revert

Vor jeder riskanten Aktion: git status und Diff prüfen.



Vor dem Commit: restore oder unstage

Dateiinhalte verwerfen

`git restore <dateiname>`

GUI: Datei → Änderungen verwerfen

Aus Staging nehmen

`git restore --staged <dateiname>`

GUI: Bereitgestellte Änderungen → –



reset: gleicher Befehl, drei sehr verschiedene Wirkungen

Modus	Commit-Zeiger	Staging Area	Arbeitsdateien
--soft	geht zurück	bleibt	bleiben
--mixed	geht zurück	wird aufgehoben	bleiben
--hard	geht zurück	wird überschrieben	werden überschrieben

--hard nur verwenden, wenn der Verlust wirklich beabsichtigt ist.



reset oder revert?

reset

Lokale Geschichte umschreiben

```
git reset --mixed HEAD~1
```

Gut für noch nicht geteilte lokale Arbeit.

revert

Neuen Gegen-Commit erzeugen

```
git revert <commit>
```

Die Historie bleibt nachvollziehbar.



Zu einem älteren Stand gehen

Nur ansehen

`git switch --detach HEAD~3`

Zurück: `git switch -`

Dort weiterarbeiten

`git switch -c <neuer-branch-name> HEAD~3`

Im Verlauf einen Commit auswählen und dort bei Bedarf einen Branch anlegen (VS Code).



Eine effiziente lokale Routine

- 1. git status — Lage klären
- 2. Diff lesen — Inhalt verstehen
- 3. Gezielt bereitstellen — Commit schneiden
- 4. Klein committen — Rückwege verkürzen
- 5. Branch wechseln oder mergen — bewusstes Ziel wählen



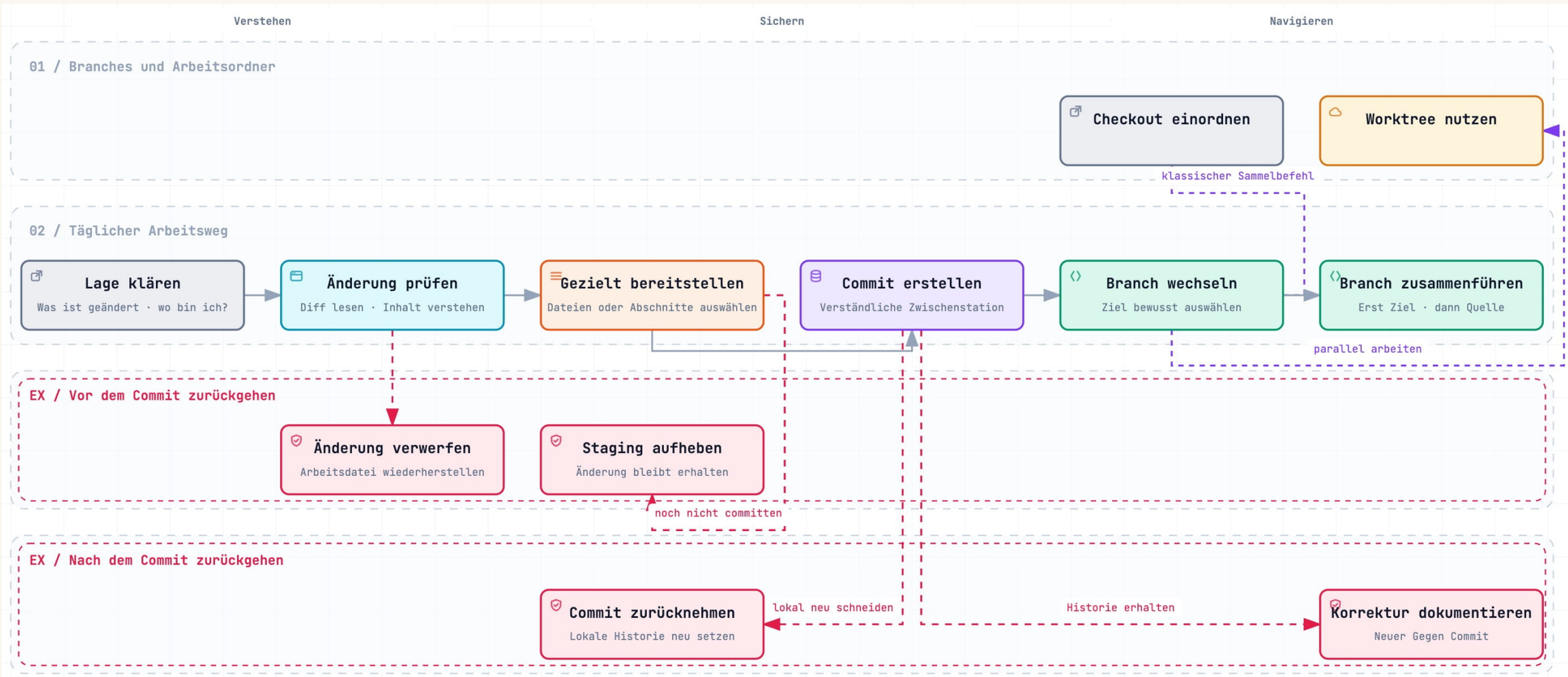
Worktrees: zwei Branches gleichzeitig offen

- `git worktree add ../<neuer-pfad> <neuer-branch-name>`
- `git worktree list`
- Nützlich für parallele Aufgaben oder einen schnellen Hotfix.
- Für den normalen Einstieg nicht nötig: Ein Arbeitsordner und saubere Branch-Wechsel reichen meist aus.



Noch erwähnenswert: stash und pull request (PR)

- `git stash` — Legt noch **nicht** commitete Änderungen vorübergehend beiseite, damit man mit einem sauberen Arbeitsverzeichnis weiterarbeiten kann; zurückholen mit
- `git stash pop` — zurückholen
- `gh pr create` — Erstellt über die GitHub CLI einen Pull Request, also die Bitte, Änderungen eines Branches zu prüfen und in einen anderen Branch zu übernehmen
- Hinweis: Ein Pull Request ist kein eigentlicher Git-Befehl, sondern eine Funktion von Plattformen wie GitHub oder GitLab. Im rein lokalen Git entspricht der technische Kern meist `git merge`





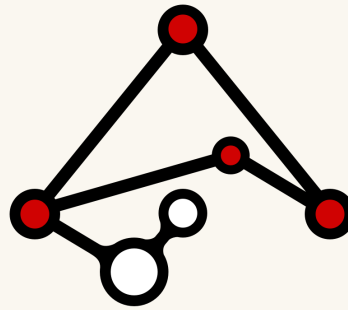
Kleine Commits. Kurze Branches. Sichere Rückwege.

Fragen?



Vielen herzlichen Dank

Dr. Adam G. Augustin



info@agametis.de - www.agametis.de
<https://github.com/agametis>